

END-TO-END MLOPS PIPELINE FOR IMAGE CLASSIFICATION

A Production-Grade Implementation with CI/CD, Containerization, Monitoring, and Artifact Publishing

YouTube Link: https://youtu.be/IZYo7U_kpU8

Group Number: 106

Github repository: https://github.com/2024aa05458-crypto/ML-OPS_Cats_Dogs

ABSTRACT

Machine Learning systems differ fundamentally from traditional software systems due to their probabilistic nature, data dependency, and continuous evolution requirements. While research frequently emphasizes model accuracy, production systems demand robustness, reproducibility, scalability, observability, and automated lifecycle management. This project implements a complete end-to-end MLOps pipeline for a binary image classification system (Cats vs Dogs) using PyTorch, MLflow, FastAPI, Docker, Docker Compose, GitHub Actions, and Docker Hub.

The project integrates model development, experiment tracking, containerization, CI/CD automation, artifact publishing, deployment orchestration, runtime monitoring, and post-deployment performance evaluation. Unlike minimal academic exercises, this implementation reflects real-world DevOps and MLOps patterns including automated smoke testing, container registry integration, service health verification, logging middleware, and inference metrics tracking.

The objective of this report is to document architectural decisions, theoretical foundations, implementation details, operational considerations, deployment mechanisms, monitoring strategies, and future scalability pathways. The work demonstrates how machine learning systems can be engineered with software engineering rigor to achieve maintainability and reproducibility.

TABLE OF CONTENTS

1. Introduction
2. Motivation and Industry Context
3. Background: DevOps to MLOps Evolution
4. Literature Review
5. Problem Statement
6. Dataset and Data Engineering
7. Model Architecture Design
8. Training Strategy and Optimization
9. Experiment Tracking and Reproducibility
10. Inference Service Architecture
11. Containerization Theory and Implementation
12. CI/CD Pipeline Design
13. Artifact Publishing and Registry Management
14. Deployment Strategy (Docker Compose)
15. Smoke Testing and Release Validation
16. Runtime Monitoring and Observability
17. Post-Deployment Model Performance Tracking
18. Security Considerations
19. Scalability Analysis
20. Reliability Engineering
21. Risk Assessment
22. Cost Analysis
23. Reproducibility and Version Control
24. Comparison with Enterprise MLOps Platforms

- 25.Limitations
- 26.Future Work
- 27.Conclusion
- 28.References
- 29.Appendices

1. INTRODUCTION

Machine learning systems today power applications in healthcare, finance, transportation, and digital platforms. However, the majority of ML projects fail to transition successfully from experimental notebooks to reliable production systems. The primary cause is not model inaccuracy but operational complexity.

Traditional ML development focuses on:

- Dataset preparation
- Model selection
- Hyperparameter tuning
- Performance optimization

Production ML systems must additionally address:

- Environment consistency
- Model reproducibility
- Deployment automation
- Monitoring and observability
- Continuous validation
- Drift detection
- Governance

This discipline is collectively referred to as **Machine Learning Operations (MLOps)**.

This project implements an end-to-end MLOps pipeline centered around a convolutional neural network (CNN) for binary image classification.

2. MOTIVATION AND INDUSTRY CONTEXT

2.1 Why MLOps Matters

Research by Gartner estimates that over 80% of ML projects never reach production. The gap between experimentation and deployment is known as the “last mile problem” in ML.

The primary challenges include:

- Non-reproducible experiments
- Inconsistent environments
- Manual deployment processes
- Absence of monitoring
- Lack of model lifecycle governance

2.2 Industry Adoption

Major cloud providers now provide MLOps platforms:

- AWS SageMaker
- Google Vertex AI
- Azure ML
- Kubeflow

However, understanding underlying principles is essential before adopting managed platforms.

This project recreates core enterprise MLOps features using open-source tools.

3. BACKGROUND: DEVOPS TO MLOPS EVOLUTION

3.1 DevOps Foundations

DevOps emerged to unify development and operations teams. Core principles include:

- Continuous Integration (CI)
- Continuous Deployment (CD)
- Infrastructure as Code (IaC)
- Automated Testing
- Monitoring

3.2 Differences Between DevOps and MLOps

Feature	DevOps	MLOps
Determinism	Deterministic	Probabilistic
Testing	Functional	Statistical + Functional
Artifacts	Binaries	Models + Data
Drift	Rare	Common
Lifecycle	Stable	Continuous retraining

MLOps extends DevOps with data-centric and model-centric workflows.

4. LITERATURE REVIEW

4.1 Hidden Technical Debt in ML Systems (Sculley et al., 2015)

Highlights complexity sources in ML pipelines including:

- Data dependencies
- Configuration management
- Feedback loops

4.2 MLflow Paper (Zaharia et al., 2018)

Introduced structured experiment tracking and model registry concepts.

4.3 Google MLOps Whitepaper (2021)

Defines three maturity levels:

1. Manual ML
2. Automated ML pipelines
3. Continuous Training

This project corresponds to Level 1.5–2 maturity.

5. PROBLEM STATEMENT

The objective is to:

1. Train a CNN to classify cat and dog images
2. Track experiments
3. Containerize the inference service
4. Implement CI pipeline
5. Push image to container registry
6. Deploy using Docker Compose
7. Monitor inference performance
8. Implement post-deployment evaluation

6. DATASET AND DATA ENGINEERING

6.1 Dataset Structure

Organized using directory-based class structure:

data/

train/

cats/
dogs/
val/
cats/
dogs/

PyTorch's ImageFolder maps class names to indices automatically.

6.2 Preprocessing Steps

1. Resize to 224×224
2. Convert to Tensor
3. Normalize pixel values

Mathematically:

If x is raw pixel value in $[0,255]$:

$$x_{\text{normalized}} = (x / 255 - \text{mean}) / \text{std}$$

Normalization improves gradient stability.

7. MODEL ARCHITECTURE DESIGN

7.1 Convolutional Neural Networks

CNNs exploit spatial locality through:

- Convolution filters
- Weight sharing
- Feature hierarchy

7.2 Layer Breakdown

1. Conv layer → Feature extraction
2. ReLU → Non-linearity
3. Pooling → Dimensionality reduction
4. Fully connected → Classification

7.3 Loss Function

CrossEntropyLoss:

$$L = -\sum y_i \log(p_i)$$

Suitable for multi-class classification.

8. TRAINING STRATEGY

Optimizer: Adam

Advantages:

- Adaptive learning rate
- Faster convergence
- Reduced manual tuning

Training was executed on CPU due to hardware constraints.

9. EXPERIMENT TRACKING

MLflow logs:

- Training loss
- Hyperparameters
- Model artifact

Benefits:

- Reproducibility
- Version control
- Comparison of experiments

10. INFERENCE SERVICE ARCHITECTURE

FastAPI chosen for:

- ASGI async performance
- Automatic documentation (/docs)
- Pydantic validation

Endpoints:

- /health
- /predict
- /metrics
- /submit-label
- /performance

11. CONTAINERIZATION THEORY

11.1 What is Docker?

Docker uses:

- Linux namespaces
- cgroups
- Union filesystem

Containers share OS kernel but isolate processes.

11.2 Layered Image Architecture

Each Docker instruction creates immutable layer:

FROM → Base

COPY → Code

RUN → Dependencies

Layer caching optimizes builds.

12. CI/CD PIPELINE DESIGN

GitHub Actions workflow includes:

1. Code checkout
2. Python setup
3. Dependency installation
4. Unit testing
5. Docker build
6. Docker push
7. Smoke test

Smoke test ensures deployment viability.

13. ARTIFACT PUBLISHING

Docker image pushed to Docker Hub.

Advantages:

- Immutable versioning
- Reproducibility
- Remote deployment

14. DEPLOYMENT STRATEGY

Docker Compose orchestrates services.

Command:

```
docker compose up -d --pull always --force-recreate
```

Ensures latest image deployment.

15. SMOKE TESTING

Smoke test verifies:

- Container runs
- Health endpoint returns 200
- API responds correctly

Failing smoke test blocks deployment.

16. MONITORING & OBSERVABILITY

Middleware tracks:

- Request count
- Latency
- Logs

Average latency:

$$\text{Avg Latency} = \frac{\sum \text{latency}_i}{N}$$

17. POST-DEPLOYMENT MODEL PERFORMANCE

Predictions stored in CSV.

True labels submitted later.

Accuracy:

$$\text{Accuracy} = \frac{\text{CorrectPredictions}}{\text{TotalPredictions}}$$

This simulates real-world drift detection.

18. SECURITY CONSIDERATIONS

- No plaintext credentials
- Secrets in GitHub Actions
- Minimal base image
- Temporary file cleanup

19. SCALABILITY ANALYSIS

Current deployment: Single node.

Scalable architecture:

- Kubernetes
- Horizontal Pod Autoscaler
- Load balancer
- Model registry

20. RELIABILITY ENGINEERING

Health checks prevent routing traffic to unhealthy containers.

Monitoring reduces mean time to detection (MTTD).

21. RISK ASSESSMENT

Risk	Impact	Mitigation
Model drift	High	Monitoring
Dependency conflict	Medium	Pin versions

Risk	Impact	Mitigation
Resource exhaustion	High	Scaling
Registry failure	Medium	Mirror registry

22. COST ANALYSIS

Cost drivers:

- Compute time
- Container storage
- Bandwidth
- Monitoring infrastructure

CPU inference cheaper but slower than GPU.

23. REPRODUCIBILITY

Ensured via:

- Git versioning
- Docker containerization
- Dependency locking
- CI automation

24. ENTERPRISE COMPARISON

Feature	This Project	Enterprise
CI/CD	Yes	Yes
Registry	Docker Hub	ECR/GCR
Monitoring	Basic	Prometheus
Scaling	Manual	Auto-scaling

25. LIMITATIONS

- No automatic retraining
- No data versioning tool
- No drift detection automation
- No Kubernetes orchestration

26. FUTURE WORK

- Kubernetes deployment
- Prometheus + Grafana
- Automated retraining
- Model registry
- Canary releases

27. CONCLUSION

This project demonstrates that MLOps principles can be applied even to simple ML models to achieve production-ready pipelines. The integration of CI/CD, containerization, artifact publishing, monitoring, and post-deployment evaluation ensures system robustness and maintainability.